MyBud
Student Assistant

A Django application for five everyday student tasks

Bikash Kushwaha

Technical project presentation | October 2026

Five tools in one student workspace

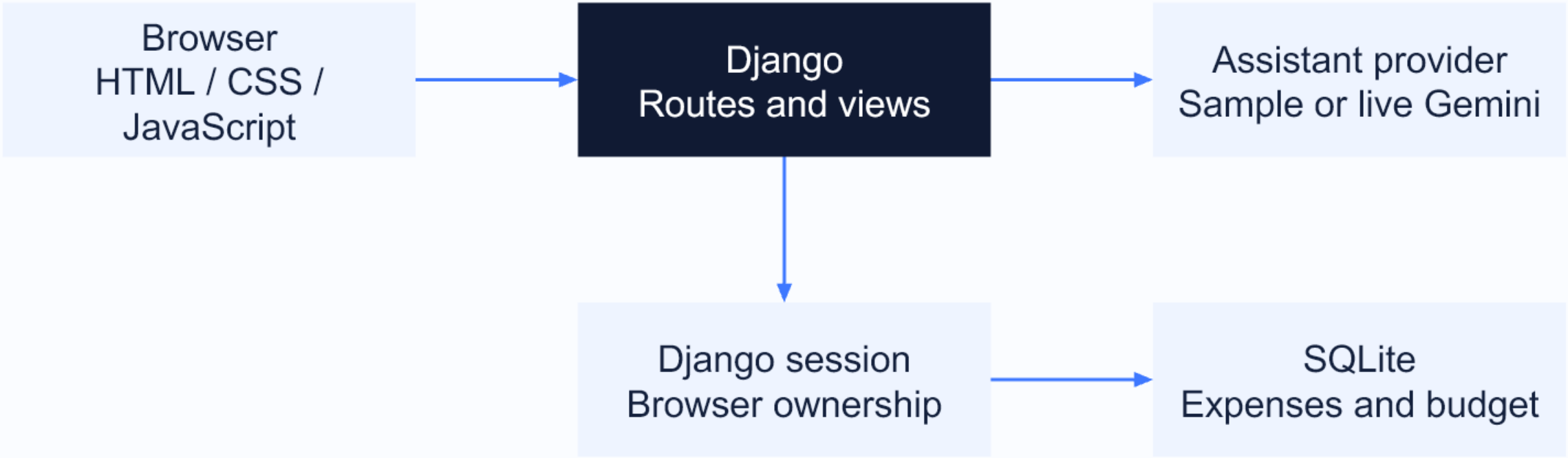
A consistent dark blue interface connects each task to a clear result.

Tool	Input	Result
Chat	A question or study prompt	Assistant response
Code review	A code snippet	Explanations and review suggestions
Recipes	Ingredients and optional image	Recipe ideas
Travel	Destination and trip duration	A day-by-day itinerary
Expenses	Category, amount and budget	Saved records and monthly totals

The four assistant tools share a provider. Expenses use the local database.

Django coordinates the application

Browser forms and JSON requests reach one server-side application.



CSRF protection and input validation apply before processing. AI credentials stay on the server.

Sample mode and live AI are explicit

Default sample mode

Works without an API key.

Returns disclosed sample content.

Also checks Python syntax locally.

No model inference runs.

Optional live Gemini

Uses the google-genai client.

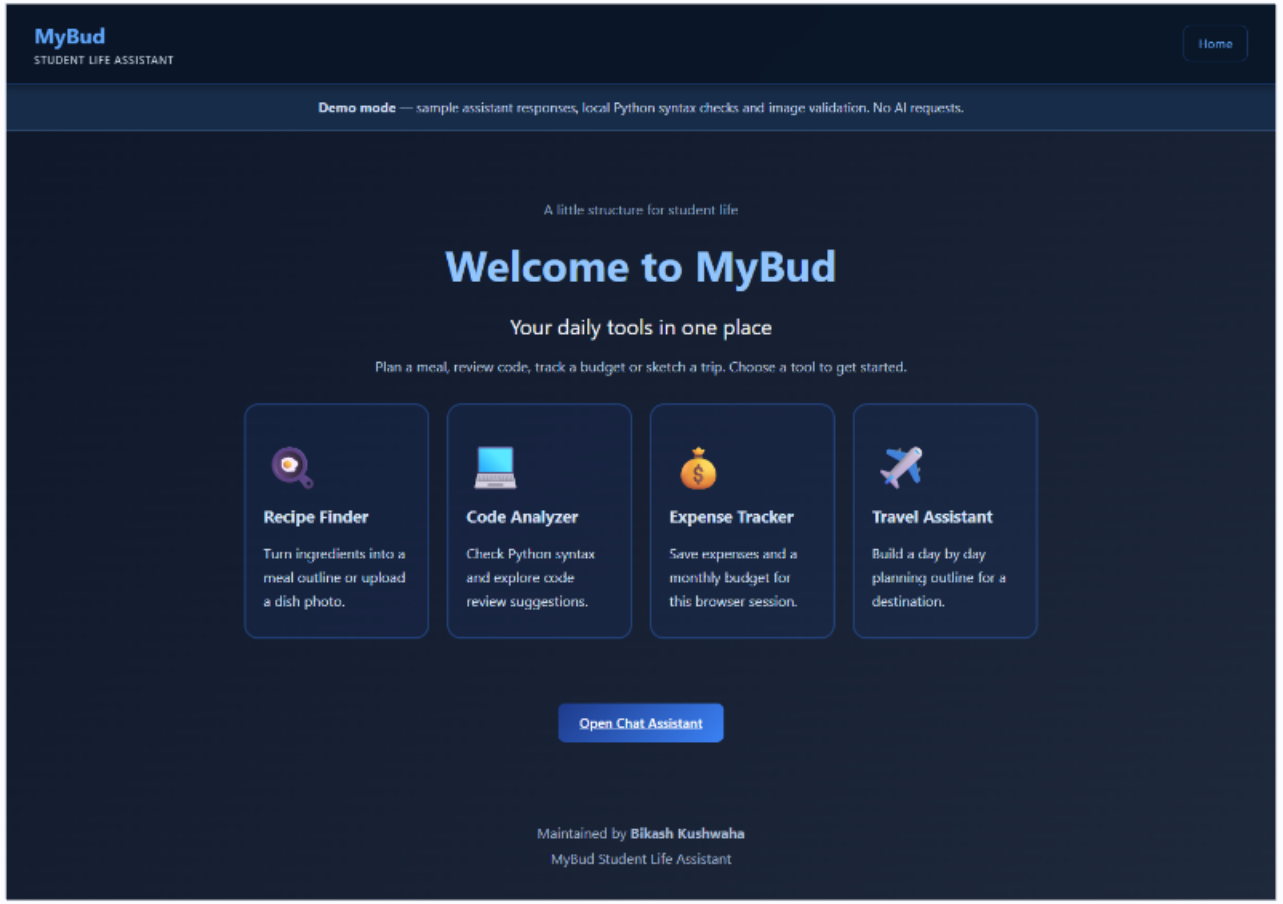
Requires a server-side API key.

The model is configurable.

Responses depend on the provider.

Assistant output needs review. A working sample does not establish live model quality.

The dashboard makes each tool discoverable



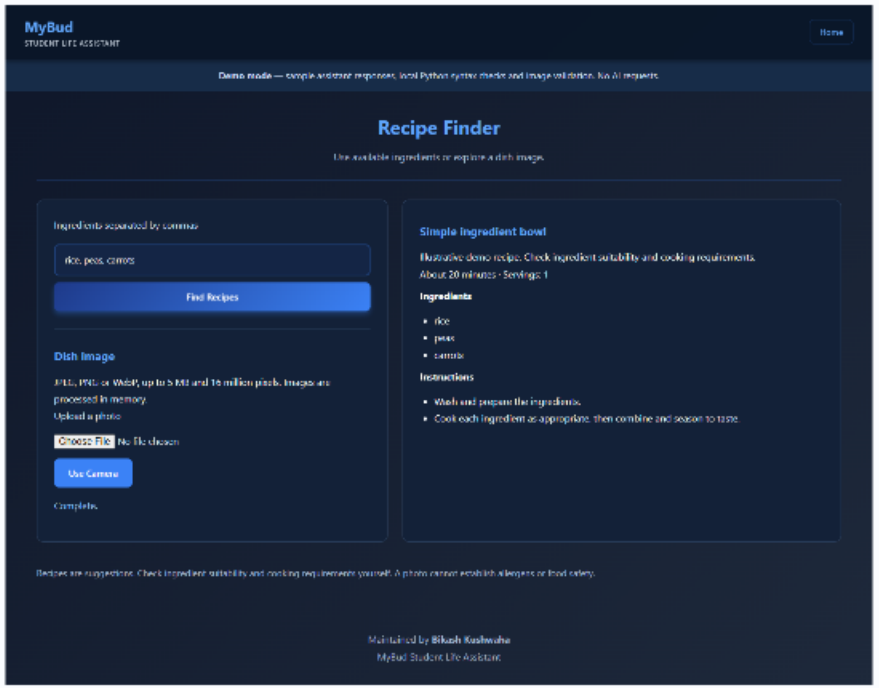
One entry point

Open a task, enter the relevant details, and review the result.

The preserved dark blue style keeps the tools visually consistent.

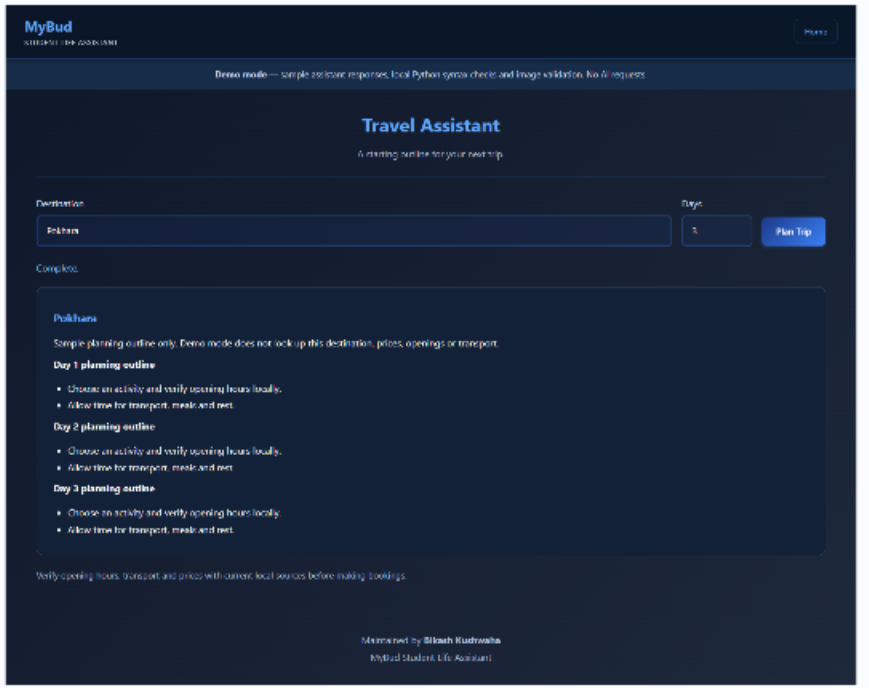
Recipes and travel show structured results

Recipe suggestions



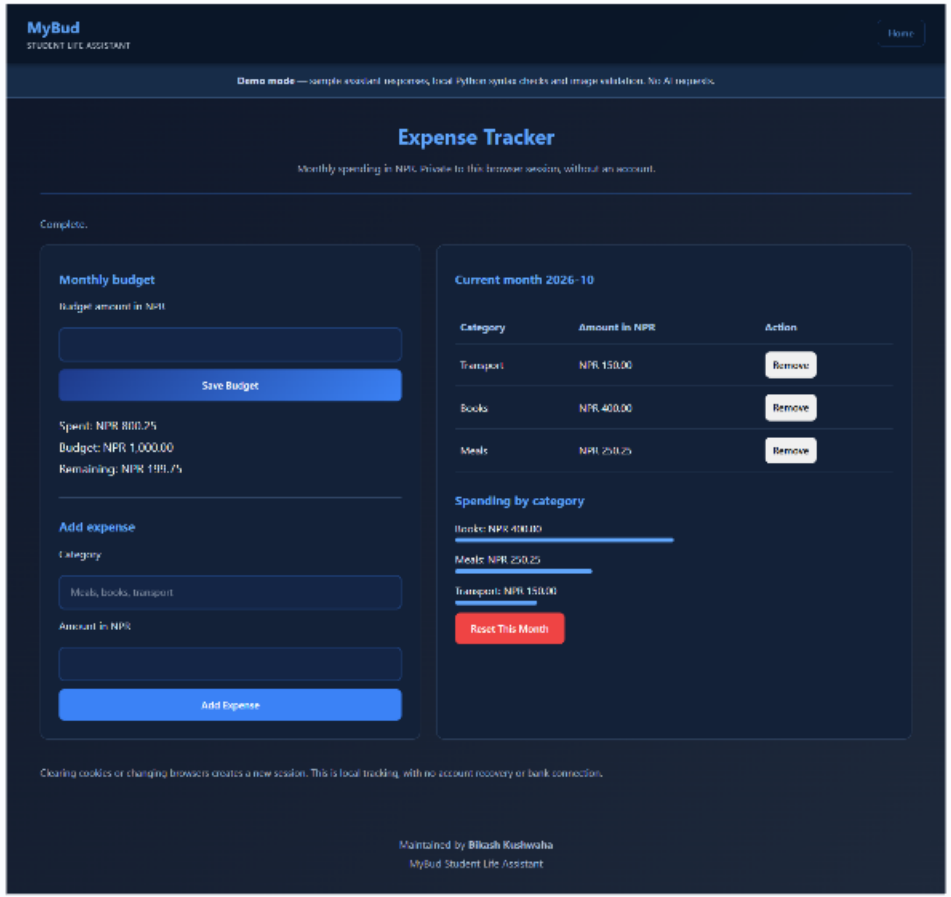
Ingredients and an optional image are validated before submission.

Travel itinerary



Destination and trip duration organize a readable day-by-day plan.

Expenses persist across page reloads



Captured NPR month summary

Measure	NPR
Budget	1,000.00
Spent	800.25
Remaining	199.75

Add and delete saved entries.
Set or reset the monthly budget.
Amounts use Decimal validation.

Ownership follows the current browser session. Account login and cross-device sync are outside scope.

Verification covers the working local flow

23 automated tests passed

Area	Recorded evidence
Backend tests	23 passing automated tests
Django checks	No system-check issues; no migration drift
Code quality	Ruff lint and format checks passed
Browser flows	All tools, uploads, expenses and session isolation
Responsive checks	Six routes at desktop and mobile widths

No authenticated live-provider call was run. Live request contracts use mocked provider responses.

Run locally with an explicit configuration

Local setup

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements-lock.txt
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py runserver
```

Configuration

MYBUD_MODE=demo is the default.

For live mode, set GEMINI_API_KEY and GEMINI_MODEL.

Copy .env.example to .env and set deployment values as needed.

Follow README.md and .env.example for the complete commands and supported settings.

A clear scope supports future development

Current limits

Sample AI content is canned.

Live AI needs external credentials.

Session storage has no account sync.

SQLite suits the local demonstration.

Next steps

Add account-based ownership.

Evaluate live responses with test prompts.

Add expense export and backups.

Review production hosting and monitoring.

Maintained by Bikash Kushwaha. Adapted from supplied starter code; see SOURCE_ACKNOWLEDGEMENTS.md.